RP8 — The source code of Revision_Deflation: what computes what, and how to compile and run it

Provenance required by Hard Rule 6 (PROJECT spacetimes, Revision_Deflation)

Prepared for Patrick Lee Nash, Ph.D.

November 13, 2026

1. What this document is for

Hard Rule 6 demands commented source code for every equation, result, figure and prediction, instructions for using it, a provenance for every computed value, and that the code be executed. This document is the map of the code of the revision: which program computes which file, which file feeds which figure and which table, how the programs are tested, and how to compile and run everything. It follows the provenance P8 of the main project, which the reader may consult for the general rules; they are the same here:

- **Rust for every computation.** All physics of the revision is computed by one Rust program, `deflation` (folder `code/deflation-rs`: 24 source files, 10324 lines, 49 unit tests). It uses the program of the main project as a library and no other library. Four of its source files are copies of modules of the main project with a listed set of changes; they are written by a script and not edited by hand (Sec. 3.1).
- **Python for an independent verification, and for tables, figures and documents.** The central results are verified a second time by a Python script that shares no code with the Rust program and uses other methods (computer algebra instead of rational arithmetic; a general-purpose integrator and library Bessel functions instead of the integrator of the program).
- **No number is typed by hand.** Every table of a document is written by a script from a data file that the program wrote; every check quoted in a document is reproduced verbatim in an appendix.

Where the code was executed. On a Linux computer (x86-64) with rustc 1.97.0, Python 3.13 (numpy 2.5.3, scipy 1.18.1, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.2), pandoc and XeLaTeX. As for the main project, the assistant could not run the Rust program on the author's Mac, where no Rust compiler is reachable from its sandbox; the program has no dependency outside the repository and should build and run on macOS unchanged.

2. How to compile and run everything on a Mac

Each block can be pasted into the Terminal as it is. A line that begins with # is an explanation and does nothing. The Rust compiler and the Python packages are installed as described in P8 of the main project, Sec. 2, Steps 1 and 2; if the main project has been run on this Mac, nothing needs to be installed. The independent check needs two Python packages more than the main project:

```
# once: the two additional Python packages (computer algebra; precise special functions)
python3 -m pip install --user sympy mpmath
```

Step 1: go to the folder of the revision and build the program.

```
# every command of the revision is run from this folder
cd "/Users/nsh/Developer/github/spacetimes/Revision_Deflation"
# are the four copied modules what vendor_eb.py writes from the sources of the main project?
```

```
# (must end with "the four copied modules are up to date";
#  without --check the script writes them again)
python3 code/scripts/vendor_eb.py --check
# compile the program (the first time also that of the main project; about a minute)
cargo build --release --manifest-path code/deflation-rs/Cargo.toml
# run the unit tests; the last lines must say "test result: ok. 49 passed"
cargo test --release --manifest-path code/deflation-rs/Cargo.toml
```

Step 2: run the fourteen commands. Each prints its checks, one per line, beginning with `[PASS]` or `[FAIL]`, and ends with a line such as `matrices: ... checks passed`. The first nine take about three and a half minutes together. The last five are the Einstein–Boltzmann calculations of the supplement RS5 and take about an hour and ten minutes on two processor cores, most of it the commands `families` and `superfluid`. Together the fourteen rewrite every file of the folder `data`.

```
# geometry, Dirac matrices, spin connection, Dirac operators          (less than a second)
./code/deflation-rs/target/release/deflation matrices
# solutions of the Dirac equations: creation and extinction          (about 12 seconds)
./code/deflation-rs/target/release/deflation modes
# gap, bound pair, matter, vacuum, temperature along the clock       (about 1.5 minutes)
./code/deflation-rs/target/release/deflation history
# can the paired state follow the deflation?                         (about 16 seconds)
./code/deflation-rs/target/release/deflation quench
# Hubble constant, bounds on the rate, the exponent under H2         (about a minute)
./code/deflation-rs/target/release/deflation cosmology
# surviving symmetries, vortex zero modes, exponents of the couplings (less than a second)
./code/deflation-rs/target/release/deflation symmetry
# exchange amplitudes, modes of the scalar                           (about a second)
./code/deflation-rs/target/release/deflation channels
# dark matter, growth of structure, drifts                           (about 10 seconds)
./code/deflation-rs/target/release/deflation matter
# the paired state from the effective action                         (about 10 seconds)
./code/deflation-rs/target/release/deflation paired
# recombination and the sound horizon on the deflating background    (about 5 seconds)
./code/deflation-rs/target/release/deflation background
# spectra of the microwave background at a fixed angle; closures     (about a minute)
./code/deflation-rs/target/release/deflation spectrum
# mock data against the rate of deflation: Fisher matrices and fits  (about 6 minutes)
./code/deflation-rs/target/release/deflation forecast
# the dark-radiation families of the main project, with deflation    (about 40 minutes)
./code/deflation-rs/target/release/deflation families
# superfluid dark matter: suppression of small-scale structure       (about 20 minutes)
./code/deflation-rs/target/release/deflation superfluid
```

(`deflation families quick` runs two of the seven families at two densities for one data set in three minutes. It is a test of the command: it writes its files to a temporary folder, which it names, and leaves the folder `data` as it is.)

Step 3: the independent verification (about 20 seconds; must end with "43 of 43 checks passed"), and the test that it can fail (about four minutes; must end with "16 of 16 deliberate errors detected").

```
python3 code/scripts/verify_deflating_independent.py
python3 code/scripts/mutation_test_independent.py
```

Step 4: symbols, tables, figures, documents.

```
# the registry of symbols (provenance RP1)
python3 code/scripts/write_symbol_registry.py
# are the numbers quoted in the running text of the documents numbers that the programs stated?
python3 code/scripts/check_numbers.py
# the tables of the documents, from the data files and from the reports of the checks
python3 code/scripts/write_doc_tables.py
# the 19 figures (written to the folder figures as .pdf and .png)
python3 code/scripts/make_figures.py
# the documents: Markdown from the sources, then LaTeX and PDF (needs pandoc and XeLaTeX)
```

```
python3 code/scripts/assemble_docs.py
bash code/scripts/build_docs.sh
# the two manuscripts (needs pdflatex with REVTeX 4.2)
python3 code/scripts/build_manuscripts.py
# the six checks of the symbols (they read the registry, the data files and the documents)
python3 code/scripts/check_symbols.py
```

Two of these scripts read documents and write reports that documents print: the report of `check_numbers.py` is quoted in RP8 and in the README, and the report of `check_symbols.py` is printed in RP1. After a change of a document the commands of Step 4 are therefore run twice, and the second run must leave every data file, report, table, figure, Markdown file and LaTeX file as it was. (The PDF files of the documents other than the manuscripts carry the time of their compilation and differ in that alone.) In the repository as it stands a single run leaves these files as they are.

If a command of Step 2 prints a line that begins with `[FAIL]`, or if the numbers in a document change when Steps 2 to 4 are repeated, something is wrong and should be reported; on another processor the last digits of floating-point results may differ, the exact checks may not.

3. The Rust program, file by file

All files are in `code/deflation-rs/src`. Every file begins with a description of the physics that it computes, with the formulas; every public function is documented.

file	lines	content
`rat.rs`	223	exact rational numbers with overflow checks; the trait that lets the geometry be computed in rational or in floating-point arithmetic
`qmat.rs`	201	exact matrices with complex-rational entries
`metric.rs`	714	the deflating metric: Christoffel symbols, Riemann, Ricci and Einstein tensors from their definitions and in closed form; Killing vectors; Lie derivative
`geometry.rs`	443	the exact geometry checks `G1` to `G7`, `G10` (Lovelock tensors) and the geodesics `G8`, `G9`
`curved.rs`	530	coordinate matrices, spin connection, Dirac operators; checks `C1` to `C8`, `D1` to `D7`
`modes.rs`	766	integration of the mode equations (commutator-free Magnus method), creation of particles, bounded solutions, modified Bessel function of complex order
`scalar.rs`	295	modes of a scalar field: exponents, static source, creation, bounded solutions
`history.rs`	274	the couplings along the clock; thresholds and events
`quench.rs`	280	time-dependent mean-field dynamics of the shell model
`cosmo_defl.rs`	512	expansion with deflation, sound horizon, age, bounds on the rate, growth of pressureless matter for the closures
`spectra_defl.rs`	611	assumption A3 and its closures; the Hubble constant at a fixed angle with the full thermal history; potentials and gaps of single modes; derivatives of the spectra
`eb_cosmo.rs`, `eb_boltzmann.rs`, `eb_spectra.rs`, `eb_fit.rs`	1891	the Einstein-Boltzmann program of the main project on the deflating spacetime: copies of four of its modules with a listed set of changes, written by `code/scripts/vendor_eb.py`
`symmetry_defl.rs`	217	test of the 28 rotations and boosts; algebra of the vortex zero modes
`cmd_symmetry.rs`, `cmd_channels.rs`, `cmd_matter.rs`, `cmd_paired.rs`	1017	the commands `symmetry`, `channels`, `matter`, `paired`
`cmd_spectra.rs`	1425	the commands `background`, `spectrum`, `forecast`, `families`, `superfluid`
`main.rs`	808	the commands `matrices`, `modes`, `history`, `quench`, `cosmology`

file	lines	content
`io.rs`, `lib.rs`	117	writing of data files and reports; list of modules

(The numbers of lines and of unit tests in this document are counted by the script that writes the tables; they are not typed.)

What is taken from the program of the main project (crate `spacetimes`, used as a library and not changed): the Dirac matrices and their checks, the routine for the internal symmetry, the exchange amplitudes, the gap equation, the bound pair, the effective action, the routines for matter, temperature and vacuum, the background cosmology without deflation, linear algebra, and from its Einstein–Boltzmann part the parameters, the dark fluids, the superfluid dark matter, the Bessel functions, the mock data, the likelihood and the Fisher matrix. The revision calls these routines with the couplings of each moment; it does not copy them.

3.1 The four copied modules

The Einstein–Boltzmann program of the main project has Newton's constant built into its units and the ordinary equation of the expansion built into its routines. On the deflating spacetimes both change (RS3, Sec. 1; RS5, Sec. 2). The main project is paused and its program is not to be changed, so the revision cannot add a parameter to it; and a library routine cannot be altered from outside. The four modules that must change are therefore *copied*:

file of the revision	copied from (main project, `code/spacetimes-rs/src`)	what is changed
`eb_cosmo.rs`	`cosmo.rs`: the structures `Background` and `Thermo`	the equation of the expansion (RS3, Eq. (1.2)); the clock of Newton's constant; the helium fraction
`eb_boltzmann.rs`	`boltzmann.rs`	Newton's constant of the moment in every source of the metric; H_e in the energy constraint; the closure "Pressure"; the gaps; the sources of the temperature
`eb_spectra.rs`	`spectra.rs`: the functions `compute` and `matter_power`	they take the deflation as an argument
`eb_fit.rs`	`forecast.rs`: the functions `minimize_linearized` and `fit`	they take the deflation as an argument

So that a copy cannot differ from its original in any place that is not listed, the copies are **not edited by hand**. The script `code/scripts/vendor_eb.py` reads the four original files and applies a list of replacements, each of which must match the original text exactly once; otherwise it stops. The head of the script states every change in words, and the head of each written file repeats the list. If the main project changes one of the four files, `vendor_eb.py --check` reports that the copies are no longer current, and running the script without `--check` either reproduces the copies from the new originals or stops at the replacement that no longer matches.

Three tests guard the copies (file `spectra_defl.rs`; checks `def-E1`, `def-B1`, `def-F1`, `def-U1`):

1. *Without deflation the copies are the program of the main project.* The spectra TT, TE, EE for $2 \le \ell \le 2500$, the matter power spectrum, σ_8, the angle and the sound horizon agree **bit for bit** with the library routines, for the fiducial model, for superfluid dark matter and for a stepped dark fluid, and for both closures.
2. *With a rate too small to matter the deflating code paths give the same numbers.* A rate of 10^{-30} (in units of 100 km/s/Mpc) makes the program execute every branch of the deflating model; potentials, sources and spectra are again identical to the last bit.

3. *With a real rate each closure is a consistent theory.* The quantity that the source of the temperature uses equals the derivative of the potentials along the solution, and the two identities between the gaps (RS5, Sec. 3.1) hold along the solution to the accuracy of a numerical derivative.

4. The checks

command	file of checks	number
`matrices`	`data/flat_matrix_checks.txt` (the checks of the main project, again)	106 of 106
`matrices`	`data/deflating_checks.txt`	47 of 47
`modes`	`data/mode_checks.txt`	7 of 7
`history`	`data/history_checks.txt`	8 of 8
`quench`	`data/quench_checks.txt`	5 of 5
`cosmology`	`data/cosmology_checks.txt`	6 of 6
`symmetry`	`data/symmetry_checks.txt`	18 of 18
`channels`	`data/channel_checks.txt`	7 of 7
`matter`	`data/matter_checks.txt`	6 of 6
`paired`	`data/paired_checks.txt`	7 of 7
`background`	`data/background_checks.txt`	2 of 2
`spectrum`	`data/spectrum_checks.txt`	8 of 8
`forecast`	`data/forecast_checks.txt`	5 of 5
`families`	`data/families_checks.txt`	4 of 4
`superfluid`	`data/superfluid_checks.txt`	2 of 2
Python script	`data/deflating_checks_independent.txt`	43 of 43
symbols	`data/symbol_checks_deflation.txt`	6 of 6
numbers of the running text	`data/number_checks_deflation.txt`	974 numbers in 29 documents, 0 not traced

Each file of checks exists also as `.csv`. A check marked “exact” is an identity verified in exact rational arithmetic; a check marked “numerical” states its accuracy. The identifiers begin with `44d-` (deflating 4+4), `71d-` (deflating 7+1) or `def-` (both); those of the Python script with `I`; those of the symbols with `RS`.

Was the independent script itself tested? Yes: twenty deliberate errors were introduced into it, one at a time (a wrong sign in a Christoffel symbol, a wrong factor in the Einstein tensor, the wrong order of a Bessel function, and so on). The script detected nineteen. The twentieth, a change of the term gprod in the equation of the expansion, cannot be detected by the comparison of Hubble constants, because that term is absorbed by the cosmological term (RS3, Sec. 2); it is tested by the symbolic check of the Einstein tensor instead. The checks of the completeness of the Killing vectors and of the Lanczos tensor were added afterwards; the second is validated by the agreement of two independent formulas (the program uses the formula for curvatures that act on pairs of directions, the script the standard formula with four terms).

The checks of the perturbation equations, of the closures, of the growth of structure and of recombination, which were added for the supplement RS5, were tested in the same way, and for them the test is itself a script: `mutation_test_independent.py` introduces sixteen deliberate errors, one at a time, into a copy of that part of the independent script (a wrong factor in an Einstein equation, in an identity between the gaps, in a growth equation, in the response of recombination and of the helium fraction to Newton’s constant, and so on), runs the copy, and records which check fails. Its report:

```
PROJECT spacetimes, Revision_Deflation: mutation test of the independent verification of the
 ↪  cosmological perturbations
generated by: code/scripts/mutation_test_independent.py

the unchanged script: 11 checks (Idef-C1, Idef-C2, I44d-P1, I71d-P1, Idef-P2, Idef-P8, Idef-P3,
 ↪  Idef-P4, Idef-P5, Idef-P6, Idef-P7), all pass

 1. energy constraint with He = Hc - a gsum instead of Hc - a gsum/2: detected by I44d-P1, I71d-P1
 2. trace equation with He h' instead of 2 He h': detected by I44d-P1, I71d-P1
 3. closed form of B with (a gsum)^2/2 instead of (a gsum)^2/4: detected by Idef-P2
 4. pressure of the agent with gprod instead of 2 gprod: detected by Idef-P8
 5. first identity between the gaps with (D - DP) instead of (D - 3 DP): detected by Idef-P3
 6. second identity between the gaps with He instead of 2 He: detected by Idef-P3
 7. growth equation of the closure "Momentum" with the source not reduced: detected by Idef-P4
 8. growth equation of the closure "Pressure" with the friction Hc instead of He: detected by
 ↪  Idef-P4
 9. numerical growth, closure "Pressure": friction reduced by gsum instead of gsum/2: detected by
 ↪  Idef-P5, Idef-P7
10. numerical growth, closure "Momentum": source not reduced: detected by Idef-P5, Idef-P7
11. system of the closure "Pressure" with a gsum instead of a gsum/2: detected by Idef-P6
12. system of the closure "Shear" with Hc instead of He: detected by Idef-P6
13. recombination on a background in which Newton's constant does not change: detected by Idef-C2
14. response of the helium fraction to Newton's constant doubled: detected by Idef-C2
15. Peebles' factor with 4 pi instead of 8 pi (an error of atomic physics): detected by Idef-C2
16. equation of the expansion with the term -gsum H doubled (integration at fixed last
 ↪  scattering): detected by Idef-C1

16 of 16 deliberate errors detected
```

Are the numbers in the text the numbers of the programs? The tables of the documents are written by a script. The running text quotes numbers by hand, and a quoted number can be mistyped or can outlive a change of the program. The script `check_numbers.py` extracts every number from the running text of the sources of all documents and of the two manuscripts (numbers with a decimal point, numbers in scientific notation, and integers from 20 on) and looks it up among the numbers that the programs *stated*: in the statements of the checks, in the generated tables, and in the descriptions at the head of the data files. A number of the text counts as traced if

- (a) a stated number equals it; or
- (r) a stated number rounds to it at the precision typed; or
- (b) it is listed in the script, with the reason, as a number that the programs do not state: a value from the literature, a definition, a count that follows from a formula of the text, a parameter of a run, or (in the error log RP9 only) a superseded value that the log quotes; or
- (c) it is typed with one significant digit in scientific notation, as a bound or a power of ten, and a stated number lies below it within one decade.

The numbers inside the data files are deliberately not used: the files hold more than a hundred thousand numbers, and almost any number is the rounded value of one of them by coincidence. The matches of the kinds (r) and (c) are weaker than those of kind (a): among several thousand stated numbers a short number can have a rounded partner by coincidence. Each number of these two kinds must therefore be *judged* by a reader, who reads the sentence and the output of the programs and enters into the script the stated number that the text means. The report `data/number_checks_deflation.txt` lists these numbers one by one, with the stated numbers that match and with the judgement, and the script fails if one of them has not been judged. Result: 974 numbers in 29 documents, 0 not traced; different numbers of the kinds (r) and (c), all judged: 10. The check is a necessary condition, not a sufficient one: it cannot tell whether the right number is quoted in the right place, and a match can be a coincidence. It found the errors R-E28 and R-E29 of the error log RP9. The stale value of R-E27 passed it, because a number of a generated table rounds to it by coincidence; that error was found by reading the program.

5. The scripts

All in `code/scripts`.

script	what it does
`vendor_eb.py`	writes the four copied modules of the Einstein–Boltzmann program from the sources of the main project (Sec. 3.1); with `--check`, tests that they are current
`verify_deflating_independent.py`	the independent verification (Sec. 4)
`mutation_test_independent.py`	introduces sixteen deliberate errors into the cosmological part of the independent verification, one at a time, and records which check detects each (Sec. 4)
`write_symbol_registry.py`, `check_symbols.py`	the registry of the symbols and its six checks (RP1)
`write_doc_tables.py`	writes every table of the documents from the data files (`provenance/generated/tables_deflation.md`)
`make_figures.py`	draws the 19 figures from the data files
`assemble_docs.py`	inserts tables, registry and program output into the document sources (`docs_src/*.src.md`)
`build_docs.sh`, `doc_header.tex`, `check_layout.py`	Markdown to LaTeX to PDF, and a check that no line, table cell or formula is wider than its box
`build_manuscripts.py`, `tex2md.py`	the two manuscripts from one source (`docs_src/manuscript/*.tex`); their Markdown versions
`check_numbers.py`	looks up every number of the running text of the documents in the output of the programs (Sec. 4)

6. Which data file feeds which figure

figure	data files	in
`fig_geodesics`	`geodesic_1671d.csv`, `geodesic_1644d.csv`	DMD, M
`fig_isometries`	`isometries.csv`	DMD, RS2, M
`fig_creation`	`creation_single.csv`, `creation_history.csv`, `creation_anisotropic.csv`, `boson_creation.csv`	DMD, M (7+1)
`fig_extinction`	`extinction_profile.csv`, `boson_bounded.csv`	DMD, M (4+4)
`fig_clock`	`coupling_exponents.csv`, `events.csv`	RS1, M
`fig_scalar`	`scalar_zero_modes.csv`, `static_source.csv`	RS1, M
`fig_kernels`	`pair_channel_kernel.csv`	RS1, M
`fig_gap_history`	`gap_history.csv`	RS1, M
`fig_binding`	`dimer_history.csv`, `dm_resonance.csv`	RS1, RS3, M
`fig_quench`	`quench_history.csv`, `quench_summary.csv`	RS1, M
`fig_cjt_history`	`cjt_history.csv`	RS4, M
`fig_matter_history`	`medium_history.csv`, `matter_saturation.csv`, `vacuum_history.csv`, `thermal_history.csv`	RS4, M
`fig_hubble`	`hubble_deflation.csv`, `hubble_mass_loss.csv`, `growth_deflation.csv`	RS3, M
`fig_bounds`	`deflation_bounds.csv`, `deflation_fisher.csv`	RS3, RS2, M
`fig_spectra`	`cmb_deflation_cl.csv`, `cmb_deflation_summary.csv`	RS5, M
`fig_closure`	`cmb_deflation_closure.csv`, `cmb_deflation_gaps.csv`, `cmb_deflation_growth.csv`, `growth_deflation.csv`	RS5, M
`fig_forecast`	`deflation_fisher.csv`, `deflation_profile.csv`, `deflation_residuals.csv`, `hubble_deflation.csv`	RS5, M
`fig_families`	`dr_deflation_profiles.csv`, `dr_deflation_joint.csv`, and `dr_profiles.csv` of the main project	RS5

figure	data files	in
`fig_sfdm`	`sfdm_deflation_transfer.csv`, `sfdm_deflation_summary.csv`	RS5

(M: the two manuscripts.) Every data file begins with lines that state which program and which command wrote it and what its columns mean.